

Sistema de radar inteligente de placas vehiculares

Detección (YOLOv11) + OCR de caracteres (CNN) + lógica difusa de sanción

Proyecto final — Inteligencia Artificial
Universidad Técnica de Ambato

Junio 2026

Resumen

Se describe un sistema de visión por computador para control de velocidad en una zona controlada (p. ej. parqueadero). El sistema: (1) detecta la región de la placa con **YOLOv11**, (2) segmenta y reconoce sus caracteres con una **red neuronal convolucional (CNN) propia** entrenada con caracteres reales, (3) estima la velocidad mediante dos líneas virtuales, (4) decide la sanción con un **sistema de lógica difusa** y (5) registra cada evento en **JSON** junto con una captura del fotograma. No se usa EasyOCR ni base de datos relacional. La precisión por carácter en el conjunto de prueba es del **91,3 %** (**93,6 %** con decodificación posicional); la **votación temporal multi-frame** recupera la placa correcta a nivel de secuencia en tiempo real.

1. Arquitectura general

El sistema procesa un flujo de video en tiempo real con la siguiente tubería:

Frame → YOLOv11 (región de placa) → Preproceso (gris+CLAHE) → Segmentación de caracteres → CNN (36 clases) → Decodificación posicional → Votación temporal → Placa

En paralelo, un módulo de fondo (*background subtraction*) mide la velocidad con dos líneas virtuales; la velocidad alimenta el sistema difuso que decide la sanción. El reconocimiento de placa corre en un **hilo separado** (`RecognitionWorker`) para no bloquear la captura.

Archivos principales.

- `cnn/modelo.py` — arquitectura CNN (`CNNPlacas`).
- `cnn/entrenar_ocr_real.py` — entrenamiento con datos reales.
- `cnn/inferencia.py` — detección + segmentación + clasificación.
- `main.py` — bucle principal, hilo de reconocimiento y votación.
- `velocidad/logica_difusa.py` — sistema difuso de sanción.
- `utils/registro.py` — registro de eventos en JSON.

2. Datos de entrenamiento

Importante: el modelo de OCR se entrena con *caracteres sueltos*, no con placas completas ni con los videos de prueba. Los videos (`videos/carros*.mp4`) se usan **únicamente para evaluar**.

El conjunto `Dataset_OCR_Placas/` contiene caracteres reales ya segmentados (imágenes PNG de 64×64 en escala de grises), organizados como `ImageFolder` en 36 clases (0–9, A–Z):

Partición	Imágenes	Uso
<code>train</code>	50 534	ajuste de pesos
<code>valid</code>	11 829	selección del mejor modelo
<code>test</code>	5 680	evaluación final

Observaciones sobre los datos.

- Proviene de placas **indias** (p.ej. KA031351, DL3CAY9324). Esto no importa para un clasificador de *caracteres*: los glifos 0–9, A–Z son universales. Sí importa el *formato*, que se valida aparte (placa ecuatoriana ABC-NNNN).
- El conjunto es **ruidoso**: algunas clases contienen recortes parciales o ruido, y los pares *O*/0 y *I*/1 son glifos visualmente idénticos. Esto fija un techo práctico de exactitud por carácter cercano al 91 %.
- Las **clases de letras** tienen menos muestras que las de dígitos (p.ej. *Q* = 397, *O* = 1016 frente a 0 = 2610), por lo que las letras se reconocen con menor confianza.

Por qué no una CRNN *end-to-end*. Un intento previo con datos *sintéticos* (tiras de placa generadas con tipografías) fracasó porque los glifos sintéticos no se parecen a los reales. Reconstruir tiras a partir de estos caracteres reaprendería además el *formato indio* de las placas. Por eso se optó por un **clasificador de caracteres** (glifos universales) más una etapa de segmentación y validación de formato.

3. Modelo clasificador

La red **CNNPlacas** es una CNN de estilo ResNet (~1,2 M parámetros) que recibe una imagen en escala de grises de 48×48 y produce una distribución sobre 36 clases.

Bloque	Salida
Stem (2×Conv3×3, BN, ReLU, MaxPool, Dropout)	$64 \times 24 \times 24$
ResBlock	$64 \times 24 \times 24$
Down1 (Conv stride 2) + ResBlock	$128 \times 12 \times 12$
Down2 (Conv stride 2) + ResBlock	$256 \times 6 \times 6$
Cabeza (AdaptiveAvgPool, FC, BN, Dropout, FC)	36

La salida son *logits* $z \in \mathbb{R}^{36}$; la probabilidad de la clase k es

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{36} e^{z_j}} \quad (\text{softmax}). \quad (1)$$

4. Entrenamiento

Script: `cnn/entrenar_ocr_real.py`.

Hiperparámetro	Valor
Tamaño de entrada	48×48 (gris)
Optimizador	AdamW ($lr = 10^{-3}$, <i>weight decay</i> 10^{-4})
Planificador	OneCycleLR
Pérdida	Entropía cruzada (<i>label smoothing</i> 0,05)
Épocas	35
<i>Batch</i>	256
Precisión mixta (AMP)	Sí (GPU RTX 5050)
Aumentación	rotación $\pm 5^\circ$, traslación/escala leve, <i>jitter</i> de brillo/contraste

La aumentación se mantiene **suave** a propósito: transformaciones agresivas (perspectiva y *shear* fuertes) sobre caracteres pequeños causaban *underfitting*. El mejor modelo (por exactitud de validación) se guarda en `models/ocr_char.pt`, incluyendo la lista de clases para garantizar el mismo mapeo índice→carácter en inferencia.

Resultados de entrenamiento.

Métrica	Valor
Exactitud de validación (mejor)	91,6 %
Exactitud por carácter en test	91,3 %
Exactitud por carácter con decodificación posicional	93,6 %

Confusiones principales (esperables): $O \leftrightarrow 0$, $7 \rightarrow 1$, $9 \rightarrow 6$, $B \leftrightarrow 8$, $Z \rightarrow 2$. La decodificación posicional (sección 5.5) elimina las confusiones *cruzadas* letra/dígito.

5. Inferencia

5.1. Detección de la placa

YOLOv11 (**best.pt**) entrega la caja de la placa de mayor confianza. Se añade un *padding* del 8 % para no cortar bordes de caracteres.

5.2. Preprocesamiento

El recorte se pasa a gris, se aplica *sharpening* si hay *motion blur* (varianza Laplaciana < 80) y se normaliza el contraste con **CLAHE**.

5.3. Segmentación de caracteres

Se usa **umbral adaptativo gaussiano** (`adaptiveThreshold`, `BINARY_INV`, ventana 21, $C = 9$) en lugar de Otsu global, que fusionaba toda la placa en un solo blob bajo iluminación irregular o *blur*. Luego:

1. apertura morfológica para quitar ruido sin unir caracteres;
2. componentes conexos $\rightarrow$ cajas candidatas;
3. filtrado por altura ($\geq 0,25H$), relación de aspecto (0,10–1,3) y área mínima;
4. filtrado de *outliers* por la mediana de alturas (elimina la caja del marco de la placa);
5. ordenamiento por la coordenada x (izquierda $\rightarrow$ derecha).

Cada caja se recorta de la imagen *en gris* (no binarizada), que es el dominio con el que se entrenó el clasificador.

5.4. Clasificación

Todos los recortes de carácter se clasifican en un **único batch** en GPU. Para cada posición se obtiene la distribución softmax $p^{(i)}$.

5.5. Decodificación posicional

La placa ecuatoriana tiene formato **ABC-NNNN**: las primeras **tres posiciones son letras** y el resto **dígitos**. Restringir el *argmax* de cada posición a su grupo elimina las confusiones cruzadas ($O \leftrightarrow 0$, $I \leftrightarrow 1$, $Z \leftrightarrow 2$, $B \leftrightarrow 8$, $S \leftrightarrow 5$):

$$c_i = \begin{cases} \arg \max_{k \in \mathcal{L}} p_k^{(i)} & i < 3 \quad (\text{letras } \mathcal{L}) \\ \arg \max_{k \in \mathcal{D}} p_k^{(i)} & i \geq 3 \quad (\text{dígitos } \mathcal{D}) \end{cases} \quad (2)$$

Solo se aplica cuando hay 6 o 7 caracteres segmentados. Finalmente, una expresión regular valida el formato `[A-Z]{3}-\d{3,4}`.

6. Votación temporal (¿por qué?)

En un sistema en tiempo real la **misma placa aparece en decenas de fotogramas**. Un solo fotograma alcanza $\sim 93\%$ de exactitud por carácter, es decir ≈ 1 carácter erróneo por blur o ángulo, lo que da una placa *completa* incorrecta. Los errores por fotograma son en gran parte **aleatorios**, así que combinarlos los cancela.

La clase `VotadorPlaca` acumula las lecturas válidas (con confianza $\geq 0,45$) y decide por **mayoría posición a posición** sobre la longitud más frecuente:

$$\hat{c}_i = \text{moda} (c_i^{(1)}, c_i^{(2)}, \dots, c_i^{(T)}), \quad (3)$$

donde $c_i^{(t)}$ es el carácter leído en la posición i del fotograma t . Se exige un mínimo de votos (por defecto 4) antes de aceptar el consenso.

Ejemplo real. Para la placa PDK-7282, fotogramas sueltos dieron HDK-7282, HLK-7282, PLK-7282...; la votación por posición recupera el **consenso correcto** PDK-7282. Sin votación, el sistema aceptaría el primer fotograma, que puede estar equivocado.

7. Velocidad y lógica difusa

La velocidad se estima con dos líneas virtuales separadas una distancia conocida d . Si el vehículo cruza A en t_A y B en t_B :

$$v [\text{km/h}] = \frac{d}{t_B - t_A} \times 3,6. \quad (4)$$

El sistema difuso (`scikit-fuzzy`) clasifica la velocidad y calcula la sanción en **horas de indisponibilidad del vehículo**. Política del proyecto:

Velocidad (km/h)	Clasificación	Sanción
0-10	Felicitación	0 h
10-20	Normal	0 h (advertencia)
> 20	Multa	X h (escala con el exceso)

Conjuntos difusos de entrada (*velocidad*): *felicitation* (trapezoidal [0, 0, 8, 11]), *normal* (triangular [9, 15, 21]), *multa_leve* (triangular [19, 24, 30]) y *multa_grave* (trapezoidal [28, 34, 40, 40]). La salida *horas* se defuzzifica por centroide; *multa_leve* mapea a horas moderadas y *multa_grave* a horas altas, de modo que la sanción crece con el exceso (p. ej. 22 km/h $\rightarrow$ 16 h; 35 km/h $\rightarrow$ 39 h).

8. Registro de eventos (JSON)

Se eliminó PostgreSQL. Cada evento relevante se agrega a `registros/eventos.json` junto con la captura del fotograma (`capturas/`). Ejemplo:

```
{
  "timestamp": "2026-06-03T15:34:51",
  "placa": "PDK-7282",
  "velocidad_kmh": 27.0,
  "clasificacion": "multa",
  "horas_indisponibilidad": 16,
  "ruta_captura": "capturas/PDK7282_20260603_153451.jpg"
}
```

9. Resultados experimentales

Se evaluó la votación temporal sobre los videos disponibles. Los videos `carros1`–`carros6` corresponden al mismo vehículo, cuya placa real (verificada con un fotograma nítido) es `PDK-7282`.

Video	Consenso	Placa real	Dígitos
carros1	PBK-7282	PDK-7282	✓
carros2	PDW-2225	(otro vehículo)	—
carros3	PDK-7282	PDK-7282	✓
carros4	PLK-7282	PDK-7282	✓
carros5	PLK-7282	PDK-7282	✓
carros6	PLK-7282	PDK-7282	✓
carros7–11	varios	(pocas lecturas)	parcial

Análisis. Los **dígitos se reconocen al 100 %**. El punto débil es la **letra central** (D), que el modelo confunde con L o B bajo *blur*: las letras se predicen con baja confianza (p. ej. $D = 0,29$, $P = 0,17$) porque el conjunto tiene menos muestras de letras y más ruido. La votación acierta cuando la lectura correcta domina (`carros3`), pero falla cuando el sesgo $D \rightarrow L$ es sistemático en ese clip.

10. Limitaciones y trabajo futuro

- **Letras poco confiables.** Reentrenar con *muestreo balanceado por clase* (`WeightedRandomSampler`) y mayor resolución (64 px) para elevar la confianza de las letras.
- **Ruido del dataset.** Auto-descartar recortes de baja confianza del propio modelo antes de reentrenar.
- **Ambigüedad intrínseca** $O/0$, $I/1$: ya mitigada por la decodificación posicional.
- **Segmentación** sensible a placas muy inclinadas; podría rectificarse la perspectiva antes de segmentar.

11. Cómo ejecutar

```
# Entrenar el clasificador de caracteres (datos reales)
.venv/bin/python cnn/entrenar_ocr_real.py

# Leer una placa de una imagen
.venv/bin/python cnn/inferencia.py <imagen>

# Pipeline completo sobre un video (ventana GUI)
.venv/bin/python main.py video videos/carros3.mp4

# Probar la logica difusa
.venv/bin/python velocidad/logica_difusa.py
```